AICTE VERIFIED STUDENT REGISTER

Database Schema and API Contract

Proposed MongoDB models, identifiers, indexes and REST endpoints for the MERN prototype.

Document: 06-DATA-API Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Contract status

Important
The schemas and endpoints in this document are proposed MERN implementation contracts, not existing APIs or official AICTE endpoints. The code agent should implement them consistently and update this document if a reviewed change is approved.

2. Data modelling rules

- Use an internal random studentRef/UUID-like identifier as the platform reference. Do not make an external identifier the global primary key.
- Keep Student separate from Enrolment: the same student may have multiple enrolment histories.
- Keep incoming UploadBatch/StagingRow separate from official Enrolment until certification and commit.
- Keep SeatLedger records unique by course, academic year and bucket scope according to the approved demo model.
- Keep policy/rule versions and source metadata; do not embed unverified quota constants in code.
- Use timestamps, actor references and explicit state transitions for sensitive changes.
- Use synthetic student identifiers only. The prototype must not store real APAAR/Aadhaar details.

3. Proposed collections and fields

Collection	Proposed key fields	Important constraints
users	emailNormalized, passwordHash, role, status, institutionId?, regionIds[], studentRef?, issuerOrgId?, createdAt, updatedAt	Unique normalized email; role enum; passwordHash excluded from all API responses.
institutions	institutionCode, name, stateCode, regionId, status	Unique institution code; region linkage is authoritative for scope filtering.
courses	institutionId, courseCode, programName, level, academicYear, approvalRef, approvedIntake, approvalFlags	Index institution/year/course; approved data needs explicit source/evidence in fixture.
students	studentRef, displayName, syntheticExternalId?, externalLinkStatus, createdAt, updatedAt	Unique studentRef; external IDs are mock/synthetic only. Do not log sensitive fields.
enrolments	studentRef, institutionId, courseId, academicYear, yearOfStudy, bucketCode, state, admittedOn, version	Indexes for student/year/state and course/year/bucket/state; do not block legitimate case detection with an overly strict student-year uniqueness rule.
seatLedgers	courseId or scopeId, academicYear, bucketCode, capacity, filled, policyVersion, updatedAt	Unique configured scope/year/bucket; capacity >= 0; 0 <= filled <= capacity invariant.
seatPolicies	academicYear, version, status, buckets[], sourceReference, authoredBy, approvedBy, effectiveAt	Draft vs published version; maker-checker separation; no default real quota percentages.
uploadBatches	batchId, institutionId, academicYear, sourceFileName, fileHash, status, counts, createdBy, certifierId?, idempotencyKey?, timestamps	Batch ID unique; upload replay policy explicit; fileHash is not an authentication credential.
stagingRows	batchId, rowNumber, rawSafeFields, parsedFields, validationStatus, lastRevalidatedAt	Unique batchId+rowNumber; purge/retention of raw upload data is configurable and documented.
validationIssues	batchId, rowNumber, ruleCode, severity, messageCode, resolutionStatus, resolutionReason?, createdBy?, resolvedBy?, timestamps	Index batch+row+severity; use safe messages and rule codes.
conflicts	conflictId, studentRef, enrolmentRefs[], conflictType, status, assignedRegionId, dueAt?, resolutionType?, resolutionReason?, timestamps	Do not expose unmasked identifiers in national lists; support case history.
consents	consentId, studentRef, verifierId, purpose, requestedFields[], approvedFields[], status, expiresAt, approvedAt?, revokedAt?	Access every request through status/expiry checks; no field beyond approvedFields.
verificationRequests	requestId, studentRef, verifierId, consentId?, purpose, requestedFields[], status, expiresAt, resultReference?, createdAt	Verifier may read own request only; student sees own requests.
credentials	credentialId, studentRef, issuerOrgId, credentialType, status, issuedAt, revokedAt?, verificationMode	Prototype QR must specify url-reference/mock mode; do not call unsigned token a signed credential.
auditLogs	sequence, timestamp, actorRef, action, entityType, entityRef, eventPayloadHash, previousHash, hash, reason?	Append-only by application convention, sequential integrity checks; not immutable against a fully privileged database attacker.
notifications	notificationId, recipientRef, type, safeMessage, readAt?, relatedEntityRef, createdAt	No secrets or full sensitive identifiers in notification text.

4. Database invariants

- A college operator can read/write only batches and staging rows assigned to their institution. Do not rely on frontend filters.
- A batch transitions through documented states such as UPLOADED, VALIDATING, NEEDS_CORRECTION, READY_FOR_REVIEW, RETURNED_FOR_CORRECTION, CERTIFIED/COMMITTING, COMMITTED or FAILED. Use one agreed enum and transition table.
- No commit while blocking errors exist. Re-check at certification/commit time, because data or policy may have changed after validation.
- Seat reservation and enrolment commit must be transactionally consistent. Use a MongoDB replica set/sharded deployment for multi-document transactions.
- If seat counts are stored as counters, reconcile them with enrolment states in tests or a maintenance check.
- State changes should record a reason and previous/current state where appropriate.

5. Proposed REST API contract

Method	Path	Access	Purpose
POST	/api/auth/login	Public, rate-limited	Authenticate; establish session.
POST	/api/auth/logout	Authenticated	Invalidate current session.
GET	/api/auth/me	Authenticated	Current user role and scope, never secrets.
POST	/api/batches	College Operator	Upload a CSV; returns batchId and status.
GET	/api/batches	Institution-scoped / relevant reviewer	List visible batches with safe counts.
GET	/api/batches/:batchId	Batch-scope permission	Batch status and summary.
GET	/api/batches/:batchId/issues	Batch-scope permission	Row issues and resolution status.
PATCH	/api/batches/:batchId/rows/:rowNumber	College Operator, own staging batch	Save allowed field corrections.
POST	/api/batches/:batchId/revalidate	College Operator, own batch	Revalidate changed rows.
POST	/api/batches/:batchId/return	College Head, own institution	Return for correction with reason.
POST	/api/batches/:batchId/certify	College Head, own institution	Recheck blockers and commit safely; conflict returns no partial commit.
GET	/api/institutions/:id/seat-ledger?academicYear=YYYY-YY	Assigned institution/officer scope	Read bucket capacity/filled/remaining plus policy version.
GET	/api/enrolments	Scoped roles	List permitted enrolment records, masked by default.
PATCH	/api/enrolments/:id/state	Authorised institution/head/case action	Validated state transition with reason.
GET	/api/conflicts	Scoped institutions/officers	List visible open/resolved cases.
GET	/api/conflicts/:id	Visible case actors	Case details and timeline.
POST	/api/conflicts/:id/resolve	Authorised case actor	Apply permitted resolution and record reason.
POST	/api/conflicts/:id/escalate	Authorised officer	Escalation event.
GET	/api/dashboard/college	College Operator/Head	Own institution summaries.
GET	/api/dashboard/aicte/national	National Officer	National aggregate view.
GET	/api/dashboard/aicte/region	Regional Officer	Only assigned region aggregate view.
GET	/api/admin/users	System Admin	List managed users with no password hashes.
POST	/api/admin/users	System Admin	Create account under permitted role/scope.
PATCH	/api/admin/users/:id	System Admin	Change allowed status/scope with audit.
POST	/api/admin/seat-policies	Admin policy maker	Create draft rule version.
POST	/api/admin/seat-policies/:id/validate	Authorised policy actor	Validate draft config.
POST	/api/admin/seat-policies/:id/publish	Different authorised approver	Maker-checker publish only after approval.
POST	/api/verification-requests	Verifier	Create request: studentRef, purpose, fields, expiry.
GET	/api/verification-requests/:id	Verifier owner / Student owner	Read only the appropriate side of the request.
POST	/api/consents/:id/approve	Student owner	Approve displayed permitted fields.
POST	/api/consents/:id/deny	Student owner	Deny pending request.

Method	Path	Access	Purpose
POST	/api/consents/:id/revoke	Student owner	Revoke active consent.
POST	/api/credentials	Authorised issuer workflow	Create a prototype credential/QR reference.
GET	/api/verify/:credentialRef	Public minimal endpoint	Return minimal demo status without open student search.
POST	/api/credentials/:id/revoke	Authorised issuer or explicit authority	Revoke permitted credential; record actor/reason.
GET	/api/audit	Scoped audit reader	Read permitted audit events.
POST	/api/audit/verify-chain	Authorised officer/admin	Run integrity check and return evidence/status.

6. Standard API behaviour

- Return JSON with stable error codes and safe messages. Never return stack traces to users.
- Recommended HTTP semantics: 400 malformed request; 401 unauthenticated; 403 insufficient permission; 404 unknown or deliberately concealed out-of-scope resource; 409 state/capacity conflict; 413 upload too large; 422 valid syntax but invalid business fields; 429 rate limit; 500 unexpected server error.
- Use pagination for list endpoints; default limits must be bounded.
- Never accept institutionId, regionId, user role or owner reference from the browser as authoritative scope. Derive it from the session and stored role.
- Mutating endpoints should be idempotent where retries could duplicate a commit, consent decision or credential issuance. Document retry semantics.

7. API response examples

Upload accepted (illustrative)

```
{
  "batchId": "BATCH-DEMO-001",
  "status": "VALIDATING",
  "receivedRows": 100
}
```

Validation issue (illustrative)

```
{
  "rowNumber": 12,
  "ruleCode": "L1_REQUIRED_FIELD",
  "severity": "ERROR",
  "message": "Required field is missing",
  "field": "courseCode",
  "resolutionStatus": "OPEN"
}
```

8. Contract discipline

- Before UI implementation, agree on request and response shapes.
- Keep route/controller logic thin; put business rules in service functions testable without React.
- Do not let the AI rename fields independently across client, API and database.
- Any contract change requires updating this document, tests and all call sites.